Database Guide

ECITY — Mobile Shop Management Web App

Version 1.0 · 2026

Setting up PostgreSQL, and the tasks you will repeat

Companion documents: PRD, Module Breakdown, Engineering Design, Deployment Guide, Database Guide

1. Who This Is For

You have not used PostgreSQL before. This guide gets it running, then covers the handful of tasks you will repeat for the rest of the project.

You will write very little SQL. The schema is TypeScript (`src/server/db/schema.ts`) and Drizzle generates the SQL for you. What you need is the *workflow* — and one habit: **read the generated SQL before you apply it.**

2. One-Time Setup

2.1 Option A — Postgres.app (recommended to start)

A normal Mac app. No containers, no Homebrew.

1. Download from `postgresapp.com` — pick the **Postgres 16** or **17** build.
2. Drag to Applications, open it, click **Initialize**. A blue elephant in the menu bar means it is running.
3. Put the command-line tools on your PATH, then open a **new** terminal:

```
sudo mkdir -p /etc/paths.d && echo \
  /Applications/Postgres.app/Contents/Versions/latest/bin \
  | sudo tee /etc/paths.d/postgresapp
```

4. Create the database and the user this project expects:

```
createdb ecity
psql -d ecity -c "CREATE ROLE ecity WITH LOGIN PASSWORD 'ecity' SUPERUSER;"
```

That matches the `DATABASE_URL` already in `.env`, so nothing else changes.

> `SUPERUSER` is fine on your laptop. Production uses a restricted role — see the Deployment Guide.

2.2 Option B — Docker (matches production exactly)

Install Docker Desktop from `docker.com`, then:

```
docker compose up -d       # starts Postgres 16, same version as production
```

`docker-compose.yml` already creates the `ecity` database, user and password, so `DATABASE_URL` works unchanged. Heavier install, one more concept, but it is the identical Postgres that runs on the server.

Either option works and you can switch later — the only difference is one line in `.env`.

2.3 Build the schema

```
npm run db:migrate    # creates the tables
npm run db:seed       # business, 2 branches, 3 roles, 3 users
npm run dev           # http://localhost:3000
```

Sign in as `admin@ecity.local` with the password in `SEED_PASSWORD` (default `ChangeMe!2026`).

2.4 Check it worked

```
psql ecity -c "\dt"                        # should list 10 tables
psql ecity -c "select email from app_user;" # should list 3 users
curl -s localhost:3000/api/health          # {"status":"ok","database":"up"}
```

3. The Five Commands You Will Actually Use

Command	What it does
`npm run db:migrate`	Applies any migration files not yet run
`npm run db:generate`	Turns schema.ts changes into a new `.sql` migration
`npm run db:seed`	Re-runs the seed. Safe to repeat — it upserts
`npm run db:studio`	Opens a browser GUI over your tables
`psql ecity`	Drops you into the SQL prompt

Start with `npm run db:studio`. Clicking through `app_user`, `role_permission` and `audit_log` teaches you the schema faster than any tutorial. Create a user in the app, then watch the row appear in `app_user` and a matching row appear in `audit_log`.

4. The Migration Workflow

This is the important part. **Never change the database by hand.**

```
1. edit  src/server/db/schema.ts        the schema lives in TypeScript
2. run   npm run db:generate            Drizzle writes drizzle/0002_xxx.sql
3. OPEN  drizzle/0002_xxx.sql           read it. every time.
4. run   npm run db:migrate             apply it
5. run   npm test                       confirm nothing broke
6. git add drizzle/ src/server/db/schema.ts && git commit
```

4.1 Why step 3 matters

Drizzle cannot tell a **rename** from a **delete plus an add**. Rename `selling_price` to `sale_price` and you may get:

```
ALTER TABLE "product" DROP COLUMN "selling_price";
ALTER TABLE "product" ADD COLUMN "sale_price" bigint;
```

That is data loss. On your laptop it costs a re-seed; on the shop's server it costs every price in the business. Reading the file takes ten seconds. If you see `DROP COLUMN` or `DROP TABLE` and did not mean it, edit the SQL by hand to `ALTER TABLE … RENAME COLUMN …` before applying.

4.2 Migrations are committed to git

The `drizzle/` folder is part of the code. Every machine and the production server replay the same files in the same order and end up with the same schema. Never edit a migration that has already been applied anywhere but your own laptop — write a new one instead.

5. Repeated Tasks

5.1 Start and stop the database

	Postgres.app	Docker
Start	Open the app, or click Start	`docker compose up -d`
Stop	Click Stop	`docker compose stop`
Is it running?	Blue elephant in the menu bar	`docker compose ps`

5.2 Reset to a completely clean database

The one you will use most while building. Wipes everything and rebuilds.

```
dropdb ecity && createdb ecity
psql -d ecity -c "CREATE ROLE ecity WITH LOGIN PASSWORD 'ecity' SUPERUSER;" \
  2>/dev/null || true
npm run db:migrate && npm run db:seed
```

Docker equivalent — `-v` deletes the data volume, which is the point here:

```
docker compose down -v && docker compose up -d
sleep 5 && npm run db:migrate && npm run db:seed
```

> `docker compose down -v` **deletes the database.** Harmless locally, fatal on a server. Never type it anywhere but your laptop.

5.3 Re-seed without wiping

The seed upserts, so it is safe to re-run at any time:

```
npm run db:seed
```

Use it after adding a permission to `src/lib/permissions.ts` — the seed syncs the `permission` table to match the code.

5.4 Add a column

```
// src/server/db/schema.ts
export const appUser = pgTable('app_user', {
  // …
  nickname: text('nickname'),          // new, nullable = safe
})

npm run db:generate && cat drizzle/0002_*.sql && npm run db:migrate
```

Adding a `NOT NULL` column to a table with rows fails unless you give it a default. Either add `.default('…')`, or add it nullable, backfill, then make it required in a second migration.

5.5 Look at data

```
npm run db:studio                       # GUI, easiest

psql ecity                              # or the SQL prompt

select id, name, email, is_active from app_user;
select * from audit_log order by created_at desc limit 10;
select r.name, count(*) from role r
  join role_permission rp on rp.role_id = r.id group by r.name;
```

5.6 Back up and restore locally

Before anything risky — a big migration, a bulk edit, an experiment:

```
pg_dump -Fc ecity > ~/ecity-backup-$(date +%F-%H%M).dump

# put it back
dropdb ecity && createdb ecity
pg_restore -d ecity ~/ecity-backup-2026-09-04-1430.dump
```

This is the same tool that backs up production, at a smaller scale. Practising here is how the real restore stops being frightening.

5.7 Which migrations have run?

```
psql ecity -c 'select * from drizzle.__drizzle_migrations order by id;'
```

Drizzle tracks applied migrations in its own table. If a migration is listed here it will not run again.

5.8 A migration failed halfway

1. Read the error — it usually names the column or constraint.
2. Check what actually landed: `psql ecity -c "\d table_name"`.
3. On your laptop, the fastest fix is almost always **reset** (§5.2).
4. If you must keep the data: fix the `.sql` file, remove its row from `drizzle.__drizzle_migrations`, and run `npm run db:migrate` again.

Never leave a half-applied migration and carry on. Fix it or reset.

5.9 Change the seed password

```
SEED_PASSWORD='something-else' npm run db:seed
```

Or set `SEED_PASSWORD` in `.env`. The seeded accounts are flagged `mustChangePassword` and **must not exist in production**.

5.10 Reset a user's password by hand

Do not write a hash into the database. Use the app: sign in as an admin and use the password reset on the user, or in development use the forgot-password flow, which prints the reset link in the terminal until email is wired up in M14.

---

6. psql Cheat Sheet

```
psql ecity              connect to the ecity database
\dt                     list tables
\d app_user             describe a table: columns, indexes, foreign keys
\di                     list indexes
\du                     list roles/users
\x                      toggle expanded output (readable wide rows)
\timing                 show how long each query took
\q                      quit
```

Ordinary SQL ends with a semicolon. Forgetting it is why the prompt turns into `ecity-#` and appears to hang — type `;` and press Enter.

---

7. Troubleshooting

Message	Cause	Fix
`ECONNREFUSED ::1:5432`	Database is not running	Open Postgres.app, or `docker compose up -d`
`role "ecity" does not exist`	The `CREATE ROLE` step was skipped	Run it — §2.1 step 4
`database "ecity" does not exist`	`createdb ecity` was skipped	Run it
`relation "app_user" does not exist`	Migrations never ran	`npm run db:migrate`
`password authentication failed`	`.env` does not match the real password	Compare `DATABASE_URL` with what you created
`port 5432 already in use`	Two Postgres installs running	Stop one, or change the port in `.env`
`column … contains null values`	Added `NOT NULL` to a populated table	See §5.4
`audit_log is append-only`	Something tried to UPDATE/DELETE audit rows	Working as intended — never edit the audit log

8. Rules

1. **Never change the database by hand** — no `ALTER TABLE` in psql, no edits in Studio to fix a schema problem. Change `schema.ts` and generate a migration, or the next deploy silently reverts you.
2. **Always read the generated SQL** before applying it (§4.1).
3. **Never edit an applied migration.** Write a new one.
4. **Back up before anything risky**, even locally (§5.6). It is 5 seconds.
5. **Money is `bigint` paise.** Never `numeric`, never `real`, never a float.
6. **Never hard-delete business records.** Documents move to Cancelled, Voided or Reversed. The audit log is append-only and enforced by a trigger.
7. `docker compose down -v` **deletes the database.** Laptop only.

9. What to Learn, in Order

You need surprisingly little to be productive here.

1. **Table, row, column, primary key** — an hour.
2. **Foreign key** — why `sale.customer_id` must point at a real customer. This is most of why the project uses Postgres and not a spreadsheet.
3. **Transaction** — several changes commit together or not at all. Central to this app: a sale must never write stock without writing payment.
4. **Index** — why an IMEI lookup stays instant across two million rows.

Safe to ignore for now: stored procedures, replication, partitioning, performance tuning, roles beyond the one you created.

One resource, about an hour: the official **PostgreSQL Tutorial**, chapters 1–3, at `postgresql.org/docs/current/tutorial.html`.

10. Local vs Production

	Your laptop	The server
Runs as	Postgres.app or Docker	Docker container beside the app
Reachable from	Only your machine	Only the app container — port 5432 is never public
Credentials	`ecity` / `ecity`	Long random password in `.env`, never committed
Backups	Manual `pg_dump` when you feel like it	Automatic, three layers, monthly restore drill
Resetting	Normal, do it freely	Never

Production is covered in `04-Deployment-Guide.md`. The important habit to carry across: **migrations are tested on staging before they go near real sales data.**